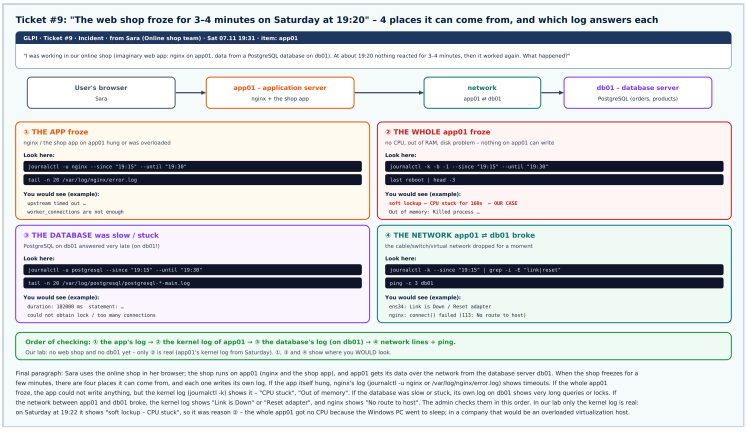
Day 16 – walkthrough: cause → check → the line that matters → next step

Project 2 · ticket #9 "the web shop froze" · db01 + PostgreSQL + nginx · every screenshot with the one line to read (yellow)

The ticket

Sara: "the web shop froze for 3–4 minutes". A shop can freeze in four places – the app, the whole server, the database, the network. Our lab had no shop yet.

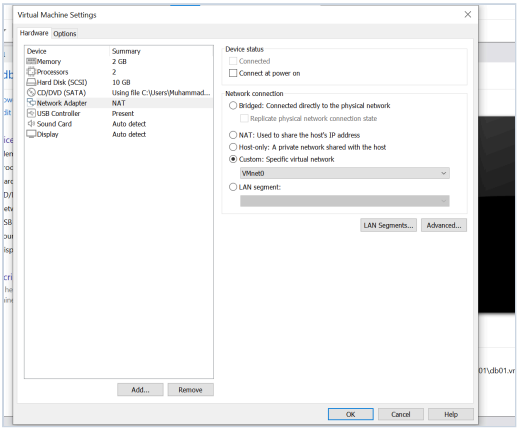


So → So we build a small shop on two servers: nginx on app01, PostgreSQL on a new server db01.

⓪-1 · New VM db01 – which network?

Why we check: db01 must sit in the same isolated lab network as app01, or it cannot talk to it.

Command: `VMware → db01 → Settings → Network Adapter`



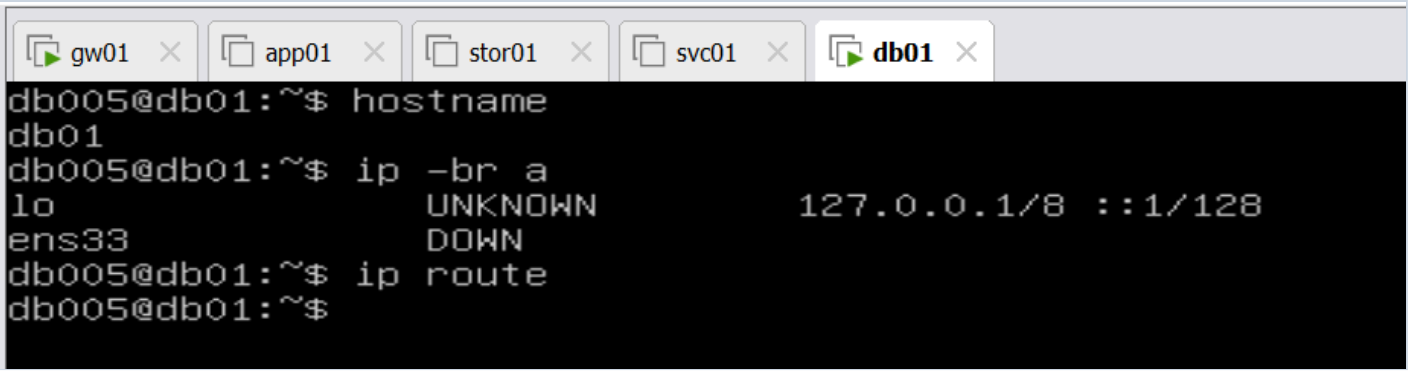
Important line: The first choice was **VMnet0 = bridged = the home network** (wrong). Correct: **Custom: VMnet2** + Connect at power on.

So → db01 is plugged into the lab switch – but VMnet2 has no DHCP, so it gets no address by itself →

⓪-2 · Does db01 have an address?

Why we check: No DHCP on VMnet2 → the installer could not set the network.

Command: `ip -br a · ip route`



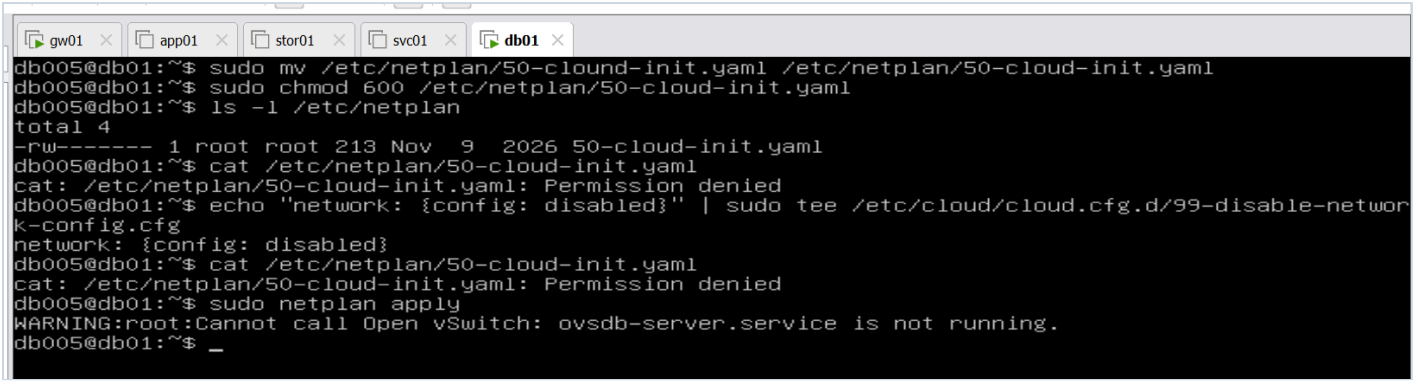
Important line: `ens33 DOWN` and `ip route` is empty – no address, no way out.

So → We must give db01 a fixed address by hand (netplan) →

⓪-3 · Fixed address with netplan

Why we check: Servers need an address that never changes; the way out must go through gw01.

Command: `/etc/netplan/50-cloud-init.yaml: 192.168.60.30/24, via 192.168.60.1, DNS 8.8.8.8 · chmod 600`



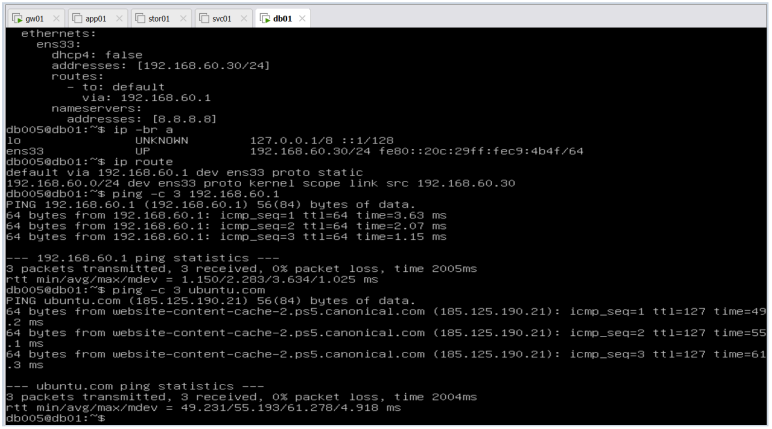
Important line: `-rw------- … 50-cloud-init.yaml` – ONE file, root only. (A typo "clound" had made a second file first.) `cat: Permission denied` is correct – use `sudo cat`.

So → Apply it and test the network →

⓪-4 · Does the network work?

Why we check: An address alone is not enough – db01 must reach gw01 (lab) and the internet (for apt).

Command: `sudo netplan apply · ip -br a · ip route · ping gw01 · ping ubuntu.com`



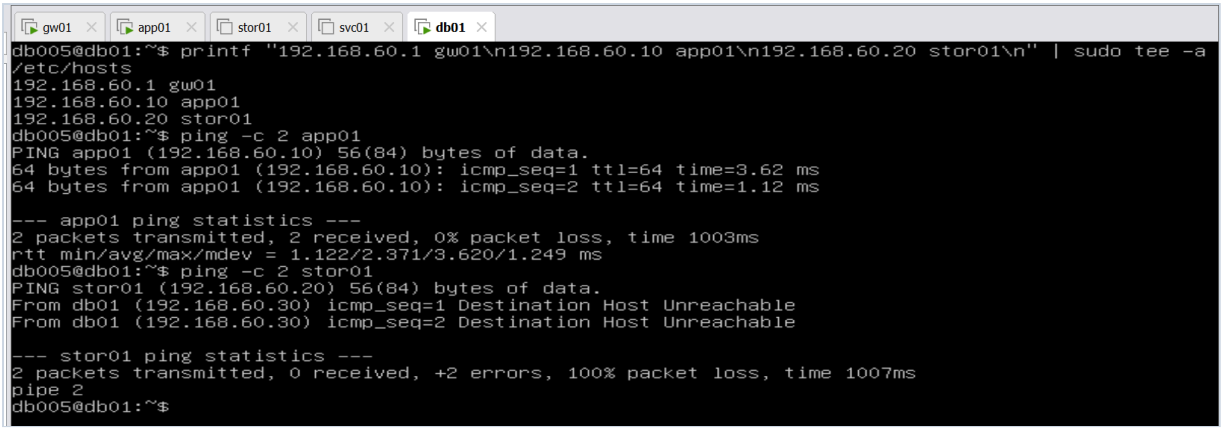
Important line: `ens33 UP 192.168.60.30/24` · `default via 192.168.60.1` · both pings `0% packet loss`.

So → db01 works by address – now give it names →

⓪-5 · Names in /etc/hosts

Why we check: Admins and apps use names (db01), not numbers.

Command: `gw01, app01: 192.168.60.30 db01 · db01: gw01, app01, stor01`



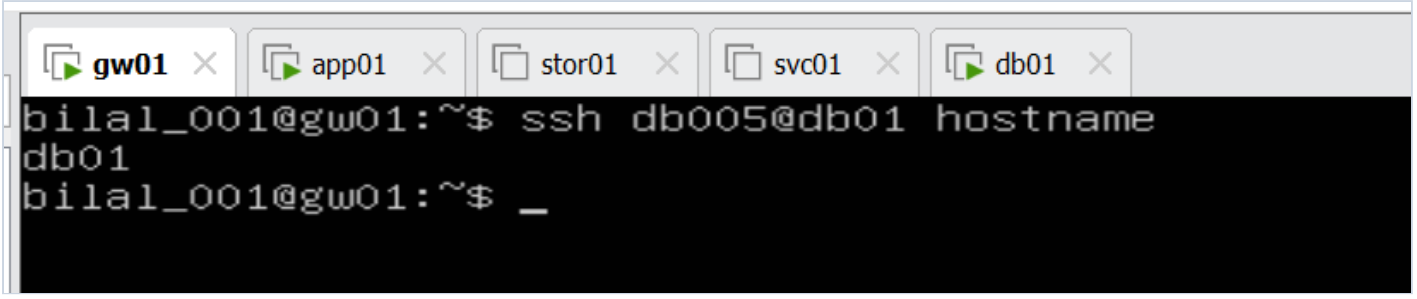
Important line: `ping app01` answers · `ping stor01` → `Destination Host Unreachable` = stor01 was simply **switched off**.

So → Lesson: "Unreachable" usually means the other machine is off, not that your settings are wrong. Next: manage db01 from gw01 →

⓪-6 · SSH key from gw01

Why we check: All servers are managed from gw01 without passwords (Phase 1).

Command: `db01: apt install openssh-server · gw01: ssh-copy-id db005@db01 · ssh db005@db01 hostname`



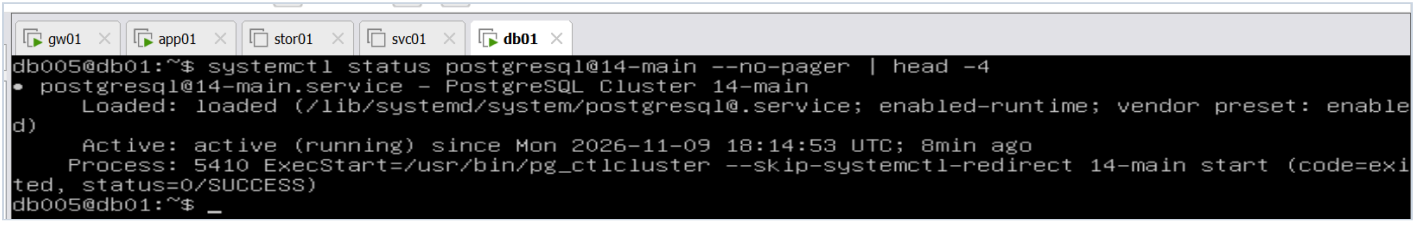
Important line: `db01` printed, no password asked.

So → db01 is a full member of the lab → install the database →

①-1 · Is PostgreSQL running?

Why we check: Before creating data, the database service must run.

Command: `systemctl status postgresql@14-main`



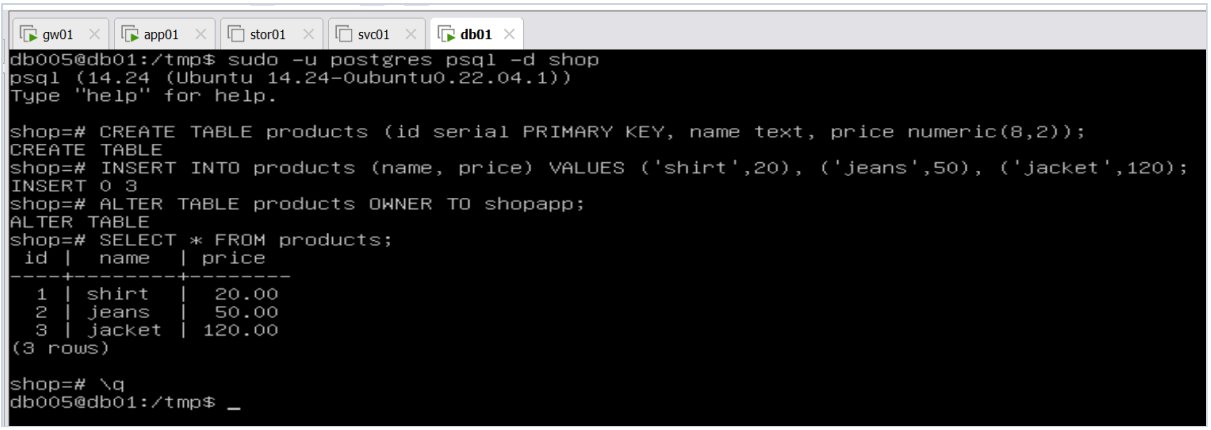
Important line: `active (running)` – the real database (cluster 14-main). `postgresql.service` shows *active (exited)* – only the umbrella, that is normal.

So → Create the shop's data →

①-2 · The first SQL

Why we check: The shop needs a table with products.

Command: `sudo -u postgres psql -d shop · CREATE TABLE · INSERT INTO · ALTER TABLE … OWNER · SELECT`



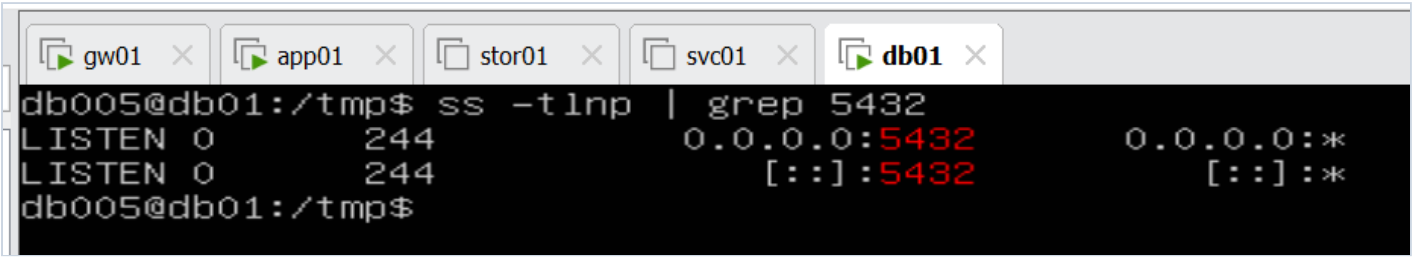
Important line: `INSERT 0 3` = 3 rows added · `SELECT * FROM products` → shirt 20.00, jeans 50.00, jacket 120.00 `(3 rows)`.

So → The data exists – but only inside db01. Open it for app01 →

①-3 · Listening on the network

Why we check: app01 is another machine – the database must listen on the network, and allow only app01.

Command: `postgresql.conf: listen_addresses = '*', log_min_duration_statement = 2000 · pg_hba.conf: host shop shopapp 192.168.60.10/32 scram-sha-256 · ss -tlnp | grep 5432`



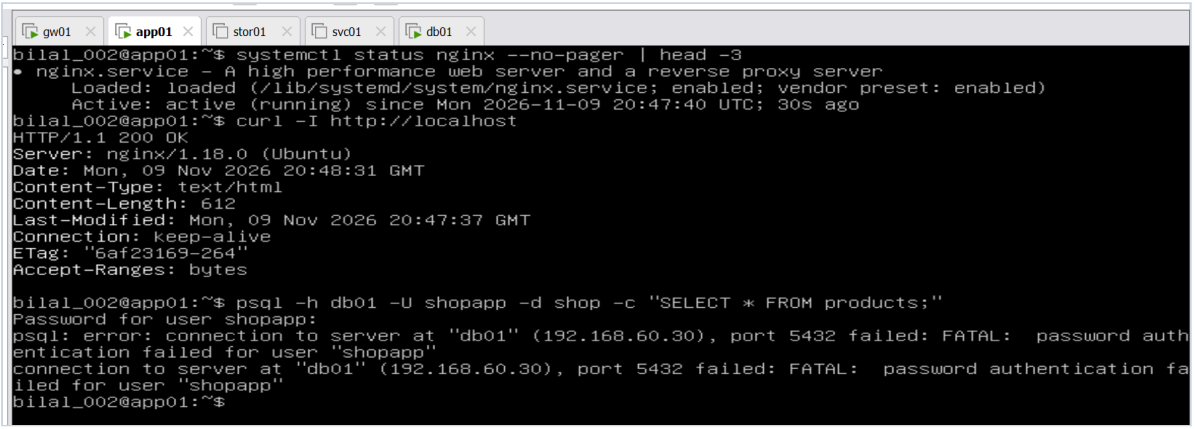
Important line: `0.0.0.0:5432` = listening on the network. pg_hba decides *who* may log in.

So → Now the other side: app01 →

②-1 · nginx and the first query from app01

Why we check: The shop's web server and a database client on app01.

Command: `apt install nginx postgresql-client · curl -I http://localhost · psql -h db01 -U shopapp -d shop -c "SELECT * FROM products;"`



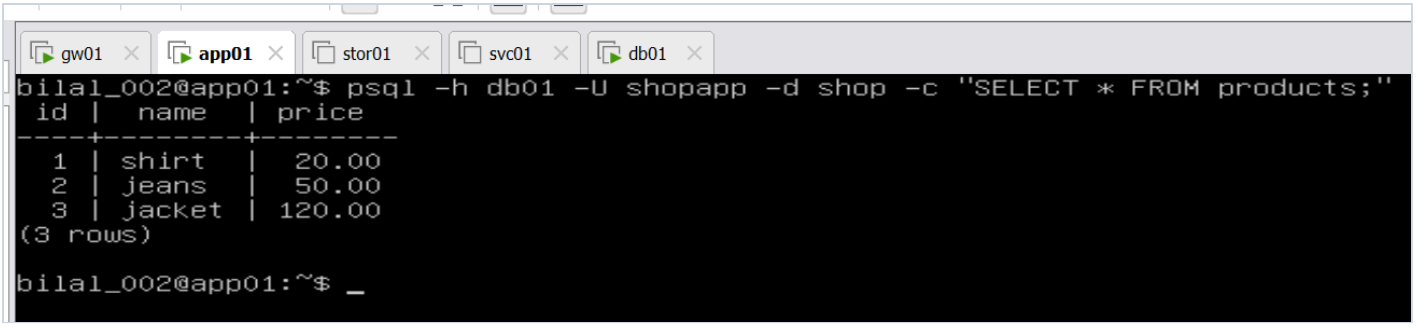
Important line: `HTTP/1.1 200 OK` = nginx works. `password authentication failed` – read left to right: app01 **reached** db01 (network ✔), db01 **knew** app01 (pg_hba ✔), only the **password** was wrong.

So → Fix the password (same in the database and in ~/.pgpass) →

②-2 · app01 → db01 works

Why we check: Prove the whole chain: app01 → network → db01 → data → back.

Command: `psql -h db01 -U shopapp -d shop -c "SELECT * FROM products;"`



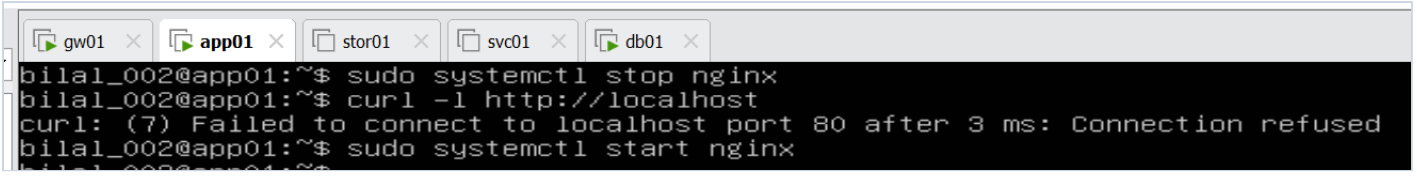
Important line: `(3 rows)` and **no password prompt** – ~/.pgpass and the database now agree.

So → The shop exists. Now ticket #9: play the four freezes one by one →

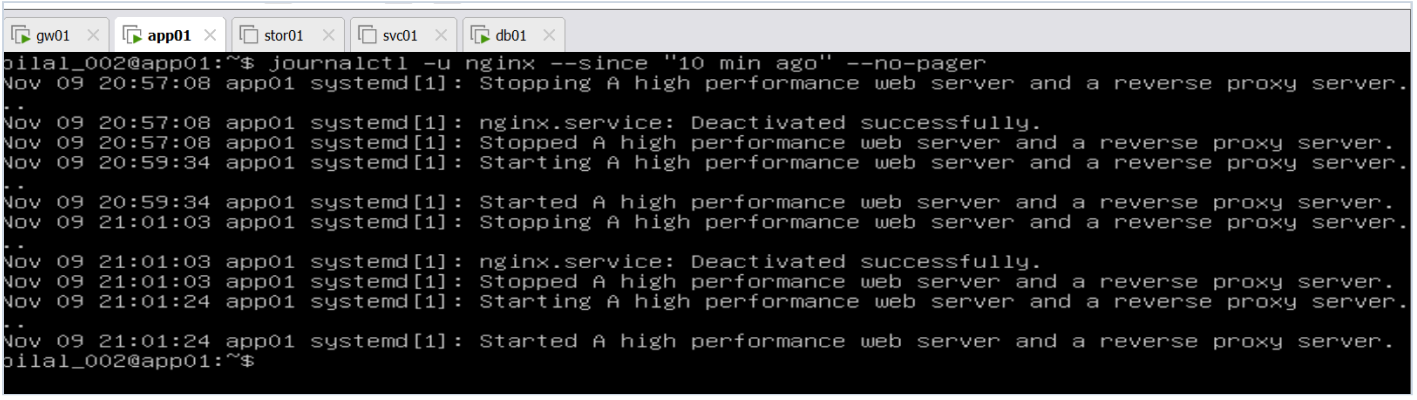
Case ① · the app froze

Why we check: If nginx stops, the shop is gone – the app's own log should say when.

Command: `sudo systemctl stop nginx · curl -I http://localhost · sudo systemctl start nginx · journalctl -u nginx --since "10 min ago"`



Important line: `Connection refused` = nothing listens on port 80 – what Sara feels.



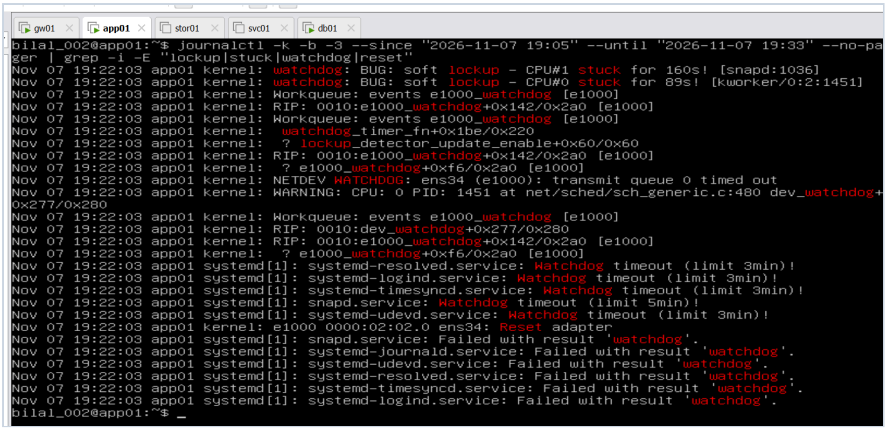
Important line: `Stopping … Stopped … Starting … Started` with the times – down 21:01:03 → 21:01:24. **The app's log answers case ①.**

So → If the app's log is clean, look one level deeper: the whole server →

Case ② · the whole server froze

Why we check: If all of app01 froze, the app could not write anything – but the kernel notices.

Command: `journalctl --list-boots (Saturday's run = -3) · journalctl -k -b -3 --since "2026-11-07 19:05" --until "2026-11-07 19:33" | grep -i -E "lockup|stuck|watchdog|reset"`



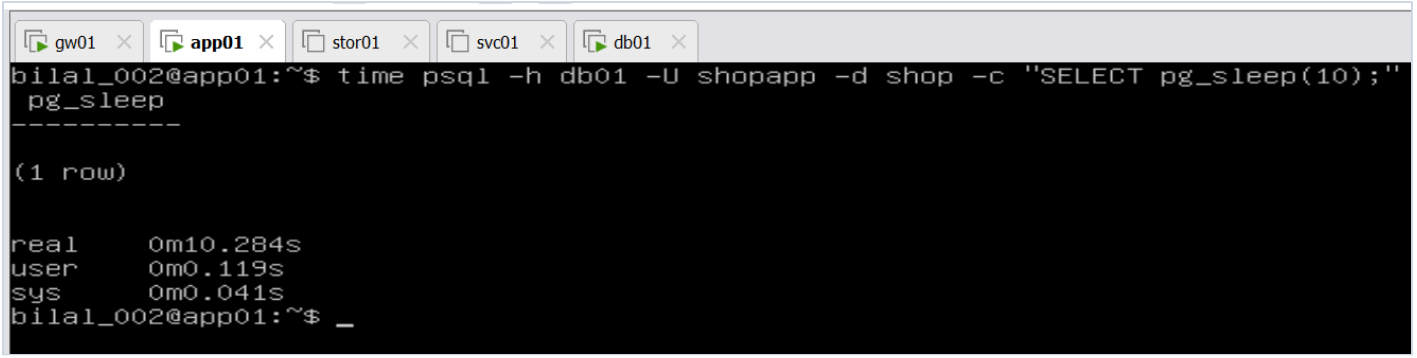
Important line: `19:22:03 … soft lockup — CPU#1 stuck for 160s!` · `NETDEV WATCHDOG … timed out` · `Reset adapter` · services `Watchdog timeout`.

So → THE REAL CAUSE: app01 got no CPU for minutes (the host PC slept). For completeness, the other two places →

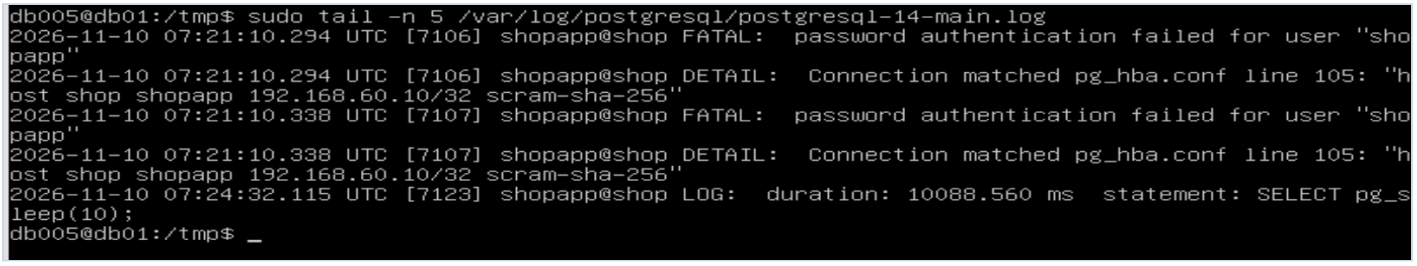
Case ③ · the database was slow

Why we check: If the database answers late, the shop waits – the database's own log names the slow query.

Command: `app01: time psql … -c "SELECT pg_sleep(10);" · db01: sudo tail -n 3 /var/log/postgresql/postgresql-14-main.log`



Important line: `real 0m10.284s` – the shop waited 10 seconds.



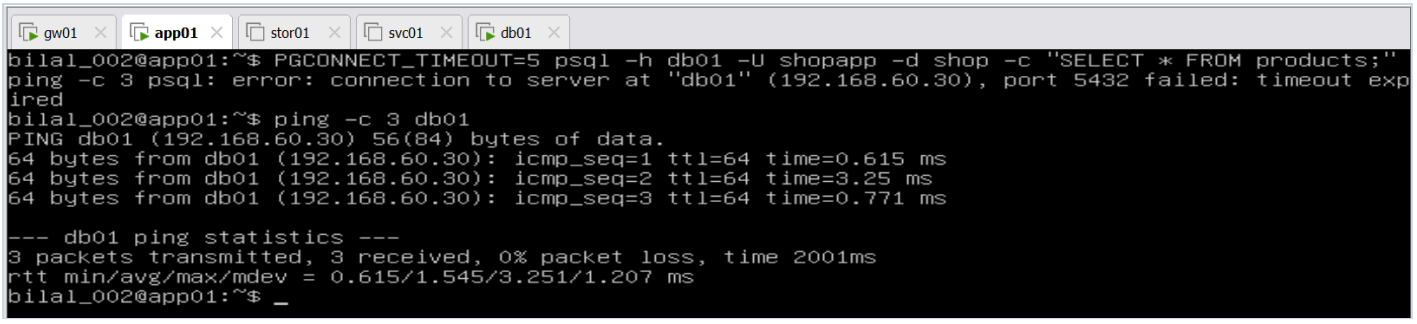
Important line: `duration: 10088.560 ms statement: SELECT pg_sleep(10);` – written because of `log_min_duration_statement = 2000`.

So → Last place: the network between them →

Case ④ · the network broke

Why we check: If the path app01 → db01 port 5432 breaks, the shop cannot reach its data.

Command: `db01: iptables -I INPUT -s 192.168.60.10 -p tcp --dport 5432 -j DROP · app01: PGCONNECT_TIMEOUT=5 psql … · ping db01 · db01: same with -D`



Important line: `timeout expired` – but `ping … 0% packet loss`. **"Ping works" does not mean "the service works"** – only port 5432 was blocked.

So → Remove the block (-D) → the 3 rows come back (step ②-2).

Final paragraph: Each check led to the next. db01 had to sit in the lab network VMnet2, which has no DHCP, so ens33 was DOWN until netplan gave it 192.168.60.30 with the way out through gw01; the pings proved the lab and the internet, /etc/hosts gave the names (stor01 was just switched off), and gw01's SSH key made db01 a full member of the lab. On db01, PostgreSQL 14-main ran, the first SQL created the products table with three rows, and listen_addresses plus the pg_hba line opened port 5432 for app01 only. On app01, nginx answered 200 OK and the first query failed with "password authentication failed" – which already proved the network and the access rule worked – until the password in the database and in ~/.pgpass agreed and the three rows came back. Then ticket #9 was checked in four places: nginx's own log showed when the app stopped (case ①), the kernel log of Saturday's run showed "soft lockup – CPU stuck for 160s" at 19:22 – the real cause, app01 got no CPU (case ②), the database's slow-query log named the 10-second query (case ③), and a blocked port 5432 gave "timeout expired" while ping still worked (case ④).